

A PROJECT REPORT

on

Current research on Pediatric Brain Tumor using Deep Learning.

**Submitted to
KIIT Deemed to be University**

In Partial Fulfilment of the Requirement for the T&T lab.

BY

AKASH ACHARYA	2105857
ANKIT SHARAN	2105863
ANNIKA SINGH	2105864
ROUNIT KUMAR	2105918



**SCHOOL OF COMPUTER ENGINEERING
KALINGA INSTITUTE OF
INDUSTRIAL TECHNOLOGY
BHUBANESWAR, ODISHA - 751024
April 2024**

Acknowledgements

We are profoundly grateful to **Suchismita Das** for her expert guidance and continuous encouragement throughout to see that this project rights its target since its commencement to its completion.

AKASH ACHARYA
ANKIT SHARAN
ANNIKA SINGH
ROUNIT KUMAR

1. INTRODUCTION

The second most common pediatric tumors in childhood after leukemia are nervous system tumors. Brain cancer is cancer that usually occurs in children, account for about 15% of pediatric cancers, sometime between birth and the age of 14 years. Brain tumors may be classified according to their source or aggression, with primary brain tumors arising in the brain, while metastatic brain tumors may occur in other parts of the body. The Health Organization (WHO), which classifies brain tumors with increasing aggressiveness in Grades I to IV, launched the most widely used grading classification scheme in 1993.

This classification is depending on region, type of tissue, degree of malignancy and various other factors. After malignancy level determinations of microscopic tested tumor cells, its grade can be assessed using cell growth rate, cells blood supply, centered dead tumor cells, and tumor cell resemblance to normal cells. The most common cancers in the pediatric age group include glioma, ependymoma, medulloblastoma, craniopharyngioma, and pinealoma. Infratentorial and supratentorial tumors arise at around the same level in infants; tumors of germ cells, teratomas, gliomas, neuroepithelial tumors (PNETs) and papillomas of the choroid plexus are recognized. Whereas, posterior fossa neoplasms (primitive neuroepithelial tumors, ependymomas, astrocytomas and hemangioblastomas) are often found in older children.

Since each type of tumor can be given special treatment, radiotherapy, surgery, chemotherapy, are some of the therapeutic options available. However, before any treatment is given, it is important to consider the nature of the brain tumor, including its size, rate of growth, and all the contributing factors mentioned earlier. Nowadays, histological and molecular diagnosis of tumors is certainly the most important thing to consider in order to understand prognosis, therapy and survival. In pediatric patients, tumor samples are also regularly subjected to genetic and protein fusion testing in addition to traditional histologic tests, providing a new degree of diagnostic accuracy. Additionally, a more precise prognosis could be possible for outcome studies that take into account molecular

subgrouping. This knowledge would certainly aid us in tailoring treatment regimens for different tumor subgroups.

Magnetic resonance imaging MRI, is the standard imaging technique for the diagnosis of brain tumors. As a non-invasive technique that is widely available in clinics, MRI provides excellent contrast between soft tissues. MRI is used to provide the most accurate tumor pathology and metabolism data in combination with other imaging methods, such as computed tomography and magnetic resonance spectroscopy.

The aim of this review is to provide a detailed summary of the current state of pediatric brain tumors studies centered on medical imaging based on deep learning. The remaining of this review study is arranged as follows: Introduces briefly pediatric brain tumor types, defines pediatric brain imaging techniques and explores the different MRI sequences to provide an inclusive background on the field. This is followed by available datasets for pediatric brain tumor modalities and data acquisition and analysis methods for human brain activity. Presents the research carried out by summarizing the scientific contributions to the field of pediatric brain tumor imaging processing and analysis. Medical and technical challenges in pediatric brain tumor can be found in this report.

Reading MRI Sequences

Because of the growth and presence of the brain tumor, one MRI sequence is not enough to properly examine the tumor. Consequently, it is a time-consuming and complicated process for radiologists to analyze image characteristics and interpret MR images. In current clinical routine, various sequences of MRIs are used to diagnose and delineate tumor compartments. Tumor orientation, heterogeneous intensity profiles, presence and overlapping intensity of the tumor tissue vary between these sequences, which can lead to several different diagnosis. It is a demanding task to distinguish between distinctive tumor types, each of which has the same features. Some tumor types, such as glioblastomas, for instance, have blurry boundaries and are difficult to discern from healthy tissues. Therefore, T1, T2, T1c (T1 with contrast), PD (proton-density weighted), dMRI (diffusion magnetic resonance imaging) and FLAIR (fluid attenuated inversion recovery) sequences are required to better diagnosis. The comparison between these modalities literally provides individuality to each type of tissue as shown in Figure 2.

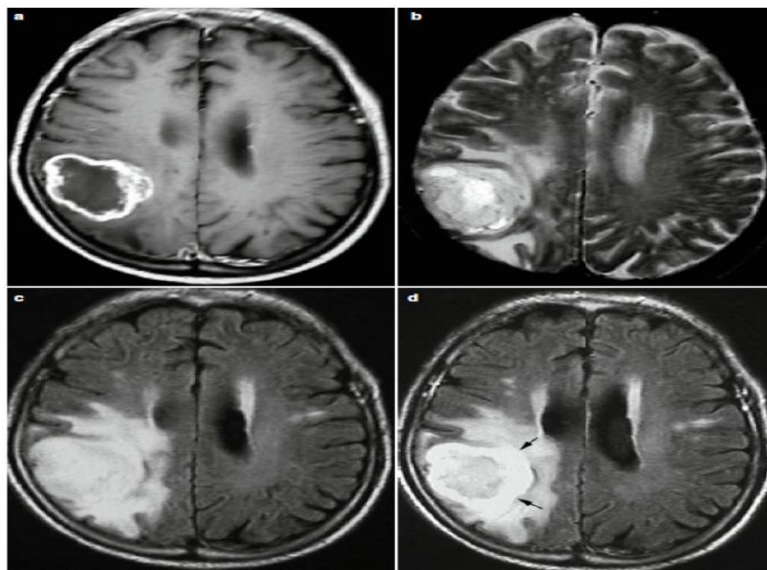


Fig 1

In the contrast enhanced images T1-weighted (gadolinium—DTPA), as the frequently used sequence for structural analysis, the tumor boundaries look brighter because the contrast agent collects there due to destruction of the blood-brain barrier in the proliferative

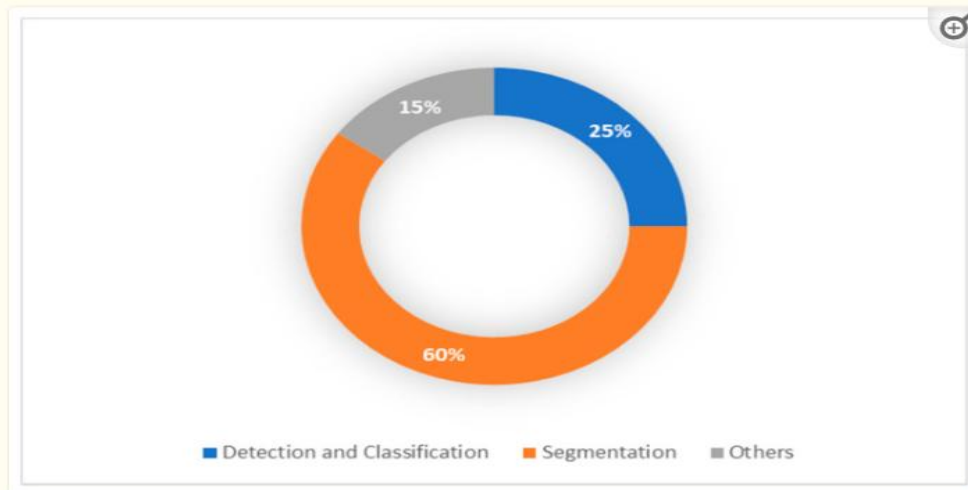
tumor zone. This is s that T1-weighted can easily realize the active tumor region whereas the region of the edema circling the tumor remains bright in the T2-weighted view. Another special sequence that tends to distinguish edema from cerebrospinal fluid (CSF) is T2 FLAIR. Astrocytomas, for instance, are usually T1-w isointense and T2-w image hyperintense. While MRI rarely classify low-grade astrocytoma, most anaplastic astrocytoma enhances by contrast agents.

MRI, however, may show some non-specific results, such as T2-weighted hyper intensity and FLAIR, in pediatric brain tumor diagnosis, which may reduce diagnostic accuracy . Nevertheless, an increase in contrast as seen in contrast enhancement MRI is a weak predictor for tumor size identification. In fact, contrast enhancement represents the permeability of a weakened blood tumor barrier to both the vascular surface region and the contrast agent. Additional diagnostic techniques capable of evaluating multiple metabolic processes, such as SPECT (computed tomography with single-photon emission), PET/CT (positron emission tomography) and MI (molecular imaging) are also effective in characterizing childhood tumors during diagnosis and follow-up after treatment.

In the clinical context, on the T2 and post-gadolinium and T1 images, the radiologist also manually defines the radiological concept of tumor boundaries by thresholding the borders between T2 and T1 contrast-enhanced lesions and the underlying normal tissues to measure the tumor's outer boundaries.

2. Current Research

From 2015 onward, the topic of deep learning to brain tumor analysis has now become dominant at different conferences and journals. However, most of these studies have primarily been focused on data from adults. Whereas, few studies focused on children brain tumors. The advancement of pediatric brain tumor MRI techniques as well as the current success of the approaches of deep learning for brain tumor diagnosis inspired us to present a thorough overview of all pediatric brain tumor regions, including detection, classification and segmentation. As can be shown in [Figure 4](#), most of the pediatric brain tumor studies focused on segmentation process due to the highly success in the segmentation process in adults MRI imaging.



[Figure 4](#)

Pediatric Brain Tumor Deep Learning-Based Studies.

2.1. Pediatric Brain Tumor Detection and Classification

Methods of deep learning have been used to detect and identify different brain abnormalities in children and fetals. This section presents existing methods for identifying and classifying pediatric brain tumor research based on deep learning. A thorough analysis of these studies can be found in the [Table 1](#).

Table 1

Pediatric brain tumor detection and classification studies based on deep learning.

Authors	Tumor Location/Type	Methodology	Modality	Dataset	Results
Arle, Jeffrey E., et al. (1997) [49]	Posterior fossa (astrocytomas, PNETs, and ependymoma)	Four back-propagation neural networks	MRS + MR + Metadata	Self-acquired dataset (33 children 6 months–14 years)	Classification accuracy rate 58–95%
Bidiwala, S. and Pittman (2004) [50]	Posterior fossa (astrocytoma, ependymoma, and medulloblastoma)	Neural networks	CT + MRI (T1WI, T2WI) + Metadata	Self-acquired dataset (33 Children)	Classification accuracy rate 72.7–85.7%
Quon, J.L., et al. (2020) [51]	Posterior fossa (diffuse midline glioma, medulloblastoma, pilocytic astrocytoma, and ependymoma)	Modified 2D ResNeXt-50-32x4d deep learning architecture	T2-weighted MRIs	Multi-institutional study (617 children)	Detection accuracy was AUROC of 0.99 Classification accuracy was 92%
Ye, Zezhong, et al. (2020) [52]	Several histologic elements of tumors of pediatric high-grade brain tumors	DHI model (DBSI + DNN)	Diffusion basis spectrum imaging (DBSI)	9 pediatric brain tumor post-mortem specimens	Overall classification accuracy rate—83.3%

Two early studies have demonstrated the capacity of neural networks to differentiate the major tumor types in the posterior fossa in children. In 1997, four neural networks were developed to incorporate MRS data with 10 tumor tissue characteristics obtained from magnetic resonance (MR) samples into patient tumor size, age and sex to increase diagnostic accuracy for 33 children in a dataset suffering from posterior fossa tumors. The collected dataset was analyzed by a neuroradiologist, then the tumor types were divided into three categories on the basis of data acquired from MR imaging. Predictions were then compared with those generated by neural networks that evaluated different variations of data. Using multiple datasets as inputs, the four proposed neural networks were capable of correctly classifying the tumor type with 58% to 95%. They reported that the neural network, which was provided with imaging data, spectroscopic data and a limited amount of clinical information

was able to accurately predict the type of pediatric posterior fossa tumor with exceptional precision. Their results also indicated that the predictive ability is improved with the increase of the input data size.

In the same context, in 2004, another neural network has been presented to classify the posterior fossa tumor. Medulloblastoma, cerebellar astrocytoma and ependymoma tumors, from 33 pediatric patients, were analyzed and used for model training and testing. The proposed network was able to accurately identify 85.7% of the tumors when all the required information was available and only 72.7% in cases with incomplete information. They also stated that the diagnosis created by the network offered precise diagnoses in both cases that the neuroradiologist conducted.

For the four most popular pediatric posterior fossa tumor pathology identification and classification, a recent study established a deep learning model based on MRI. Their dataset consisted of 617 children with four different types of posterior fossa tumors. As the basis of their multitask classifier model, they suggested a modified ResNeXt-50-32x4d architecture. With an F1 score of 0.80, the model classification accuracy exceeded 90% and the model tumor detection surpassed the area under the 0.99 ROC curve.

The authors used the innovative diffusion histology imaging (DHI) technique in another recent study which incorporates deep neural networks and diffusion base spectrum imaging (DBSI). DHI is able to classify, differentiate, and measure heterogeneous regions of pediatric high-grade brain tumors. The proposed DHI (DBSI + DNN) approach could classify six distinct types of tumor histology components with an average precision of 83.3%.

A novel genomic algorithm (GA) defines optimum design parameters in order to classify adamantinomatous craniopharyngioma in children. The efficiency enhancements for MRI-trained networks and 23% for CT-trained networks were reached by about 38% using GA

as a meta-heuristic optimizer. This resulted in 85.3% test accuracy for computed tomography (CT), 83.3% for magnetic resonance imaging (MRI) and 87.8% for composite datasets of CT and MRI.

2.2. Pediatric Brain Tumor Segmentation

For several years, brain MRI segmentation has become a growing field in computer vision. Segmentation is a fundamental phase in the quantitative study of brain imagery and the investigation of brain diseases. Most of the research, however, centered either on segmentation of adult tumors/disorder images or on normal brain segmentation for adults and/or children. As a result, there were a several studies, which have been conducted on pediatric brain tumor segmentation as summarized in [Table 2](#).

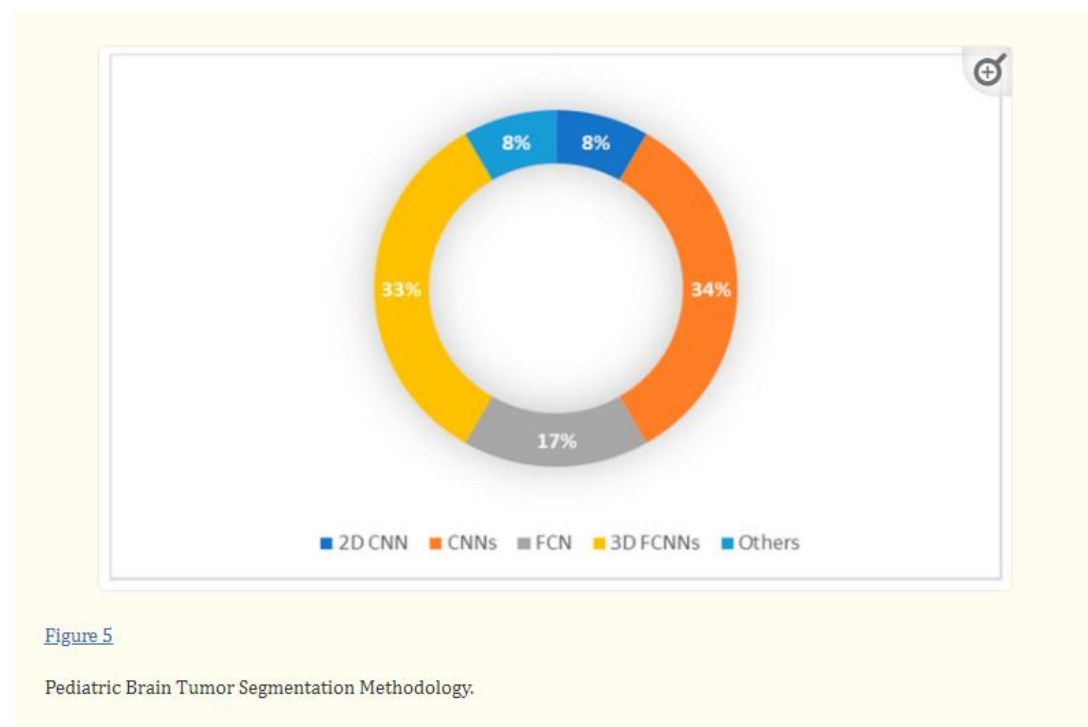


Table 2

Pediatric brain tumor segmentation studies based on deep learning.

Authors	Segmented Subject	Methodology	Modality	Dataset	Results
Zhang, Wenlu, et al. (2015) [54]	Segmenting all three types of brain tissues (CSF, GM, WM)	Four 2D CNN	T1, T2, fractional anisotropy (FA) MRI	Self-acquired (10 infants, 6–8 months of age)	Overall dice ratios CFS 83.55% GM 85.18% WM 86.37%
Cui, Zhipeng, et al. (2016) [55]	Patch-based CNN segmentation of brain structure	Three different CNNs	Manually segmented MRIs	Public dataset (CANDI neuroimaging access point 103 MRIs) small sets 4–5 MRI from each subject (6 to 17 year old age group)	Accuracy rate of 90%
Moeskops, Pim, et al. (2016) [56]	8 subjects: CB, mWM, BGT, vCSF, uWM, BS, cGM, and eCSF.	CNNs	T1-weighted and T2-weighted MRI	Self-acquired (10 images at 30 weeks, 12 images at 40 weeks, 15 images at 23 years, 20 images at 70 years)	Average dice ratios 0.87 (coronal T2w 30 weeks), 0.82 (coronal T2w 40 weeks), 0.84 (axial T2w 40 weeks), 0.86 (axial T1w 70 years) and 0.91 (sagittal T1w 23 years).

All CNNs were proposed in 2015 using multi-modality MR images to segment isointense level brain tissues. T1, T2, and FA (fractional anisotropy) multimodality images were used as input feature maps, and then segmentation maps were produced as output feature maps. The overall dice ratios total value over eight subjects achieved was 85.03%. Specifically, for the three types of brain tissues, dice ratios were generated by their proposed CNN on average over the eight subjects with 85.18% for GM, 86.37% for WM and 83.55% for CSF.

A new patch-based technique using a CNNs for automatic brain MRI segmentation was suggested by another study in 2016. Each brain MRI acquired from a public dataset is first subdivided into patches for this purpose. As a training input for the proposed CNNs, all of

these patches are then utilized. They were able to segment over 90% of the brain MRI region with their convolutional neural networks. They reported that the 90% accuracy rate outperformed other traditional approaches and methods of machine learning. With only 100,000 patches that were only extracted in four brain MRIs and trained CNNs, complex edge pixels can be successfully segmented.

A FCN was developed in 2016 in the form of segmentation of isointense phase brain MR images. They operate a convolution-pooling stream for multi-modality data from T1, T2 and FA images. They then merge them into high-layer maps to generate segmentation maps effectively. For each single modality, they often implement the FCN architecture, and then present multi-FCNs (mFCNs) for multiple modalities to integrate their complementary information effectively. In general, mFCNs, especially in the segmentation of GM and CSF, have exceeded FCNs. The average dice ratios of 0.873 for GM and 0.887 for WM, 0.855 for CSF of eight subjects were obtained by mFCNs. On the other hand, FCNs met average dice ratios of 0.861 for GM, 0.885 for WM and 0.838 for CSF.

The authors consider integrating a neural network model with an iterative graphic optimization strategy in another study to restore pixel-wise segmentation of objects from an image database with sufficient bounding box annotations. The suggested DeepCut model iteratively updates the set goals of the CNN model and utilizes a fully connected conditional random field (CRF) to regularize segmentation. The DeepCut model performs well in terms of accuracy relative to a model trained under complete supervision, and hence, greatly reduces the annotation effort required for analysis. The authors have also proposed various DeepCut models and associated them with a simplistic approach to weak supervision in CNN training. These models had overall mean in DSC (%) as follows: CNNnaïve (74.0), DCBB (86.6), DCPS (90.3), and CNNFS (94.1). An average DSC improvement of 12.6% for brain segmentation has also been reported.

A further model based on CNN, transfer learning and constructed 3D image formulation from 3D volumes was suggested in 2017. They simply stack successive 2D slices of a 3D volume in order to create a set of 2D “color” images; these 2D images reflect the input of a pre-trained FCN-based VGG network. On two types of brain MR images

(MRBrainS13 and NeoBrainS12), the proposed model has been evaluated. The suggested model segment the neonatal brain precisely into various tissues on the NeoBrainS12 dataset. For all conducted experiments, their results are based on the dice coefficient: CoGM (0.79–0.87), BGT (0.89–0.93), UWM (0.91–0.95), BS (0.76–0.86), CB (0.91–0.94), Vent (0.85–0.88), and CSF (0.82–0.89). The suggested model ranked the second best of the 38 methods submitted for adults on the MRBrainS13 challenges. Their results based on the dice coefficient were GM (86.03), WM (89.29), and CSF (82.44) on T1 sequences whereas on T1, T1-IR, and FLAIR sequences they were GM (85.40), WM (88.98), and CSF (84.13).

A 3D semantic tissue segmentation model, which based on multi-stream FCNN and context-guided 3D, was developed to map all volumetric data directly to its volume-wise labels. In conjunction, a multi-scale deep supervision has been developed to mitigating the possible gradient issue of disappearing during training. The average dice overlap coefficient (DOC) model validation on the iSeg-2017 dataset achieved: 0.916 for GM, 0.896 for WM and 0.954 for CSF.

In the context of the isointense phase of brain image segmentation, a multi-modality CC-3-D-FCN model was proposed in 2019. They integrate coarse layer information with a dense layer information in order to improve the segmentation efficacy of their model, and extra convolutions layers are also used to solve the bias of the signal problem. As reported, in terms of both segmentation accuracy and time cost, their proposed approach outperforms all comparable models on the same filed. Segmenting efficiency was obtained by CC-3-D-FCN in (DC) with: 0.9190 for WM, 0.9401 for GM and 0.9610 for CSF.

Authors focusing on an ensemble DCNNs for multimodality MRI for the isointense phase of brain image segmentation have introduced three different models. Their study is the first to use an ensemble of three-dimensional convolutional neural networks to propose annotations within images. The way to measure the level of agreement of a group of predictors is a significant advantage. This is particularly useful for assessing the segmentation's reliability at the voxel level and recommending local corrections in areas where the ensemble is uncertain about the prediction. Prediction uncertainty, measured as the opposite of predictor agreement within the

ensemble, is strongly associated with segmentation errors, according to their findings. For this purpose, three different models have been implemented. The first method, called EarlyFusion-Single, is a semi dense network with an early fusion of multi-modal images. The second model, the EarlyFusion Ensemble, comprises a group of ten EarlyFusion CNNs trained in various subjects. The third model, the LateFusion Ensemble, is a set of ten semi dense CNNs, each conducting a late fusion of modalities in various paths and trained with distinct subjects. In the iSEG-2017 challenge, the success of the proposed solution was assessed. Their methods ranking first or second among the 21 participating teams for most of the metrics.

The authors evaluated both LiviaNET and HyperDense-Net models for neonatal brain imaging to segment neonatal brain tissue types at levels of equal age. The HyperDense-Net dual-modality achieved the highest mean test DSC values of the studied segmentation methods, reaching 0.94/0.95/0.92 for the types of tissue, as their results showed. For all tissue types, in the analysis of T2 weighted images the single modality LiviaNET, was higher than in the analysis of T1 weighted images with mean DSC values: 0.90 for WM, 0.90 for GM and 0.88 for CSF.

Brain_tumor_detection.ipynb

```
import warnings
warnings.filterwarnings('ignore')
import numpy as np
import matplotlib.pyplot as plt
import os
import math
import shutil
import glob
import cv2
import imutils
import seaborn as sns
from sklearn.utils import shuffle
from keras.preprocessing.image import ImageDataGenerator, load_img
from keras.layers import Conv2D, MaxPool2D, Dropout, Flatten, Dense, BatchNormalization, GlobalAveragePooling2D
from keras.models import Model, Sequential
import keras
```

Data Loading

```
#Unzip data
!unzip '/content/brain_tumor_dataset.zip'
```

```
#Count images in the two folders: yes (Tumor) and no (Healthy)
root = '/content/brain_tumor_dataset'
dict_img = {}
for dir in os.listdir(root):
    dict_img[dir] = len(os.listdir(os.path.join(root, dir)))
dict_img
```

```
{'no': 98, 'yes': 155}
```

So we have 155 Brain MRI images with a tumor and 98 healthy ones.

```
#Plot some MRI images
plt.figure(figsize=(10, 10))
plt.subplot(2, 2, 1)
plt.imshow(load_img(os.path.join('yes', os.listdir('yes')[0])))
plt.title('yes')
plt.subplot(2, 2, 2)
plt.imshow(load_img(os.path.join('yes', os.listdir('yes')[1])))
plt.title('yes')

plt.subplot(2, 2, 3)
plt.imshow(load_img(os.path.join('no', os.listdir('no')[0])))
plt.title('no')
plt.subplot(2, 2, 4)
plt.imshow(load_img(os.path.join('no', os.listdir('no')[1])))
plt.title('no')
```

```
[5]: Text(0.5, 1.0, 'no')
```

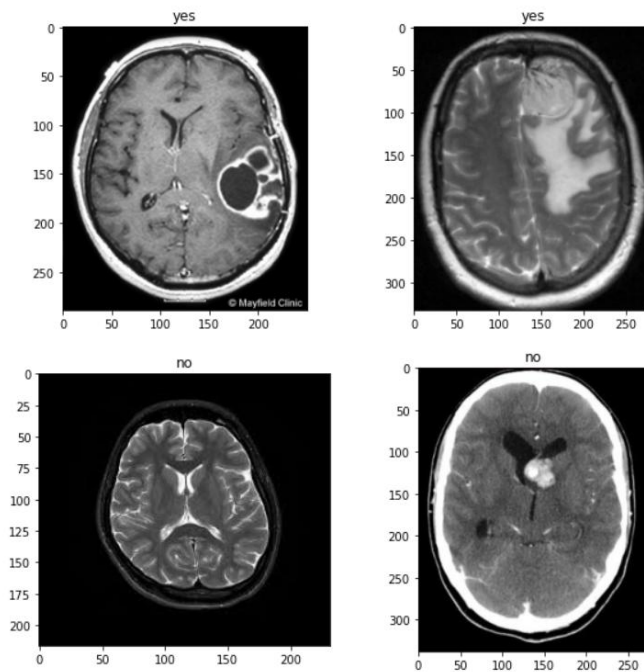


Image Augmentation

As much as our training set contains more data, the model's accuracy will improve. Data augmentation is a technique used to increase the amount of data by adding slightly modified copies of already existing data.

```
[6]: #Define a function for image augmentation
def augment_data(file_dir, n_generated_samples, save_to_dir):
    data_gen = ImageDataGenerator(rotation_range=10,
                                   width_shift_range=0.1,
                                   height_shift_range=0.1,
                                   shear_range=0.1,
                                   brightness_range=(0.3, 1.0),
                                   horizontal_flip=True,
                                   vertical_flip=True,
                                   fill_mode='nearest',
                                   rescale= 1/255
                                   )

    for filename in os.listdir(file_dir):
        image = cv2.imread(file_dir + '/' + filename)
        # reshape the image
        image = image.reshape((1,)+image.shape)
        save_prefix = 'aug_' + filename[:-4]
        i=0
        for batch in data_gen.flow(x=image, batch_size=1, save_to_dir=save_to_dir,save_prefix=save_prefix, save_format='jpg'):
            i += 1
            if i > n_generated_samples:
                break

[7]: #Create augmented images folders
#L = ['aug_train', 'aug_test', 'aug_val']
name = 'aug_data'
if not os.path.exists("./"+name):
    os.mkdir("./"+name)
    for dir in os.listdir(root):
        os.makedirs("./"+name+"/"+dir)
else:
    print(f"{name} Already exists")

[8]: #Augment data for the examples with the Label 'yes' in the training set
augment_data(file_dir='./brain_tumor_dataset/yes',n_generated_samples=8, save_to_dir='./aug_data/yes')
#Augment data for the examples with the Label 'no' in the training set
augment_data(file_dir='./brain_tumor_dataset/no', n_generated_samples=12, save_to_dir='./aug_data/no')
```

Split data into train, validation and test sets

```
[9]: #Count images in the two folders: yes (Tumor) and no (Healthy) in the folder of augmented images
root = './content/aug_data'
dict_img = {}
for dir in os.listdir(root):
    dict_img[dir] = len(os.listdir(os.path.join(root, dir)))
dict_img

[9]: {'no': 1273, 'yes': 1395}

[10]: #Define a function that creates new folders for the train, test and val sets and append random pictures to them based on the split percentage
def create_folders(name, perc):
    if not os.path.exists("./"+name):
        os.mkdir("./"+name)
        for dir in os.listdir(root):
            os.makedirs("./"+name+"/"+dir)
            for img in np.random.choice(a=os.listdir(os.path.join(root, dir)), size=(math.floor(perc*dict_img[dir])), replace=False):
                Src = os.path.join(root, dir, img)
                Dest = os.path.join("./"+name, dir)
                shutil.copy(Src, Dest)
                os.remove(Src)
    else:
        print(f"{name} Already exists")

[11]: #Create the training set
create_folders('train', 0.7)

[12]: #Create the test set
create_folders('test', 0.15)

[13]: #Create the validation set
create_folders('val', 0.15)

[14]: #Define a function that counts images in the folders: yes (Tumor) and no (Healthy)
def count_img(folder):
    dict_img = {}
    for dir in os.listdir(folder):
        dict_img[dir] = len(os.listdir(os.path.join(folder, dir)))
    return dict_img
```



```
[16]: #Count images in the test set
count_img('test')

[16]: {'no': 190, 'yes': 209}

[17]: #Count images in the validation set
count_img('val')

[17]: {'no': 190, 'yes': 209}

[18]: #Define a function for image augmentation
# def augment_data2(file_dir, n_generated_samples, save_to_dir):
#     data_gen = ImageDataGenerator(rescale = 1/255)

#     for filename in os.listdir(file_dir):
#         image = cv2.imread(file_dir + '/' + filename)
#         # reshape the image
#         image = image.reshape((1,)+image.shape)
#         save_prefix = 'aug_' + filename[:-4]
#         i=0
#         for batch in data_gen.flow(x=image, batch_size=1, save_to_dir=save_to_dir,save_prefix=save_prefix, save_format='jpg'):
#             i += 1
#             if i > n_generated_samples:
#                 break

[19]: # #Augment data for the examples with the Label 'yes' in the test set
# augment_data2(file_dir='./test/yes',n_generated_samples=8, save_to_dir='./aug_test/yes')
# #Augment data for the examples with the Label 'no' in the training set
# augment_data2(file_dir='./test/no', n_generated_samples=12, save_to_dir='./aug_test/no')

[20]: # #Augment data for the examples with the Label 'yes' in the validation set
# augment_data2(file_dir='./val/yes',n_generated_samples=8, save_to_dir='./aug_val/yes')
# #Augment data for the examples with the Label 'no' in the training set
# augment_data2(file_dir='./val/no', n_generated_samples=12, save_to_dir='./aug_val/no')

[21]: # #Count images in the training set
# count_img('aug_train')

[22]: # #Count images in the training set
# count_img('aug_test')

[23]: # #Count images in the training set
# count_img('aug_val')
```

Image Preprocessing ¶

```
[24]: #Define a function that crop tha brain contour
def crop_brain_contour(image, plot=False):

    #Convert the image to grayscale, and blur it slightly
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    gray = cv2.GaussianBlur(gray, (5, 5), 0)

    thresh = cv2.threshold(gray, 45, 255, cv2.THRESH_BINARY)[1]
    thresh = cv2.erode(thresh, None, iterations=2)
    thresh = cv2.dilate(thresh, None, iterations=2)

    #Find contours in thresholded image, then grab the largest one
    cnts = cv2.findContours(thresh.copy(), cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    cnts = imutils.grab_contours(cnts)
    c = max(cnts, key=cv2.contourArea)

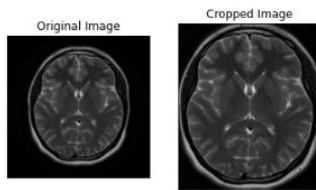
    #Extreme points
    extLeft = tuple(c[c[:, :, 0].argmin()][0])
    extRight = tuple(c[c[:, :, 0].argmax()][0])
    extTop = tuple(c[c[:, :, 1].argmin()][0])
    extBot = tuple(c[c[:, :, 1].argmax()][0])

    #Crop new image out of the original image using the four extreme points (left, right, top, bottom)
    new_image = image[extTop[1]:extBot[1], extLeft[0]:extRight[0]]

    if plot:
        plt.figure()
        plt.subplot(1, 2, 1)
        plt.imshow(image)
        plt.tick_params(axis='both', which='both', top=False, bottom=False, left=False, right=False, labelbottom=False, labeltop=False, labelleft=False, labelright=False)
        plt.title('Original Image')
        plt.subplot(1, 2, 2)
        plt.imshow(new_image)
        plt.tick_params(axis='both', which='both', top=False, bottom=False, left=False, right=False, labelbottom=False, labeltop=False, labelleft=False, labelright=False)
        plt.title('Cropped Image')
        plt.show()

    return new_image
```

```
[25]: #Example
ex_img = cv2.imread('no/1 no.jpeg')
img = crop_brain_contour(ex_img, plot=True)
```



```
[26]: #Define a function that load data
def load_data(dir_list, image_size):

    # Load all images in a directory
    X = []
    y = []
    image_width, image_height = image_size

    for directory in dir_list:
        for filename in os.listdir(directory):
            image = cv2.imread(directory+'/'+filename)
            image = crop_brain_contour(image, plot=False)
            image = cv2.resize(image, dsize=(image_width, image_height), interpolation=cv2.INTER_CUBIC)
            # normalize values
            image = image / 255.
            # convert image to numpy array and append it to X
            X.append(image)
            # append a value of 1 to the target array if the image
            # is in the folder named 'yes', otherwise append 0.
            if directory[-3:] == 'yes':
                y.append([1])
            else:
                y.append([0])

    X = np.array(X)
    y = np.array(y)

    # Shuffle the data
    X, y = shuffle(X, y)

    print(f'Number of examples is: {len(X)}')
    print(f'X shape is: {X.shape}')
    print(f'y shape is: {y.shape}')

    return X, y
```

```
[27]: #Load training data
IMG_WIDTH, IMG_HEIGHT = (240, 240)
X_train, y_train = load_data(['./content/train/yes', './content/train/no'], (IMG_WIDTH, IMG_HEIGHT))

Number of examples is: 1867
X shape is: (1867, 240, 240, 3)
y shape is: (1867, 1)
```

```
[28]: #Define a function that plot images
def plot_sample_images(X, y, n=40):
    for label in [0,1]:
        # grab the first n images with the corresponding y values equal to label
        images = X[np.argwhere(y == label)]
        n_images = images[:n]

        columns_n = 10
        rows_n = int(n/ columns_n)

        plt.figure(figsize=(10, 8))

        i = 1 # current plot
        for image in n_images:
            plt.subplot(rows_n, columns_n, i)
            plt.imshow(image[0])

            # remove ticks
            plt.tick_params(axis='both', which='both',
                            top=False, bottom=False, left=False, right=False,
                            labelbottom=False, labeltop=False, labelleft=False, labelright=False)

            i += 1

        label_to_str = lambda label: "Yes" if label == 1 else "No"
        plt.suptitle(f"Brain Tumor: {label_to_str(label)}")
        plt.show()
```

```
[29]: #Plot samples from the training set
plot_sample_images(X_train, y_train)
```

Brain Tumor: No



Brain Tumor: Yes



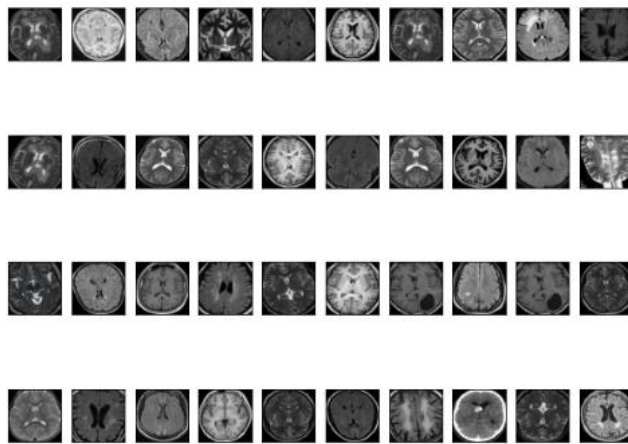
We can notice that the noise has been deleted and some images look alike with some slight changes resulted from the data augmentation

```
[30]: #Load test data
      IMG_WIDTH, IMG_HEIGHT = (240, 240)
      X_test, y_test = load_data(['/content/test/yes', '/content/test/no'], (IMG_WIDTH, IMG_HEIGHT))
      Number of examples is: 399
      X shape is: (399, 240, 240, 3)
      y shape is: (399, 1)

[31]: #Load validation data
      IMG_WIDTH, IMG_HEIGHT = (240, 240)
      X_val, y_val = load_data(['/content/val/yes', '/content/val/no'], (IMG_WIDTH, IMG_HEIGHT))
      Number of examples is: 399
      X shape is: (399, 240, 240, 3)
      y shape is: (399, 1)

[32]: #Plot samples from the validation set
      plot_sample_images(X_val, y_val)
```

Brain Tumor: No



Brain Tumor: Yes



CNN Model

```
[33]: #Build our model
model = Sequential()

model.add(Conv2D(filters = 16, kernel_size = (3,3), activation = 'relu', input_shape = (248, 248, 3)))
model.add(Conv2D(filters = 32, kernel_size = (3,3), activation = 'relu'))
model.add(MaxPool2D(pool_size = (2,2)))

model.add(Conv2D(filters = 64, kernel_size = (3,3), activation = 'relu'))
model.add(MaxPool2D(pool_size = (2,2)))

model.add(Conv2D(filters = 128, kernel_size = (3,3), activation = 'relu'))
model.add(MaxPool2D(pool_size = (2,2)))

model.add(Dropout(rate = 0.25))

model.add(Flatten())
model.add(Dense(units = 64, activation = 'relu'))
model.add(Dropout(rate = 0.25))
model.add(Dense(units = 1, activation = 'sigmoid'))

model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 128, 128, 16)	448
conv2d_1 (Conv2D)	(None, 128, 128, 32)	4640
max_pooling2d (MaxPooling2D)	(None, 64, 64, 32)	0
conv2d_2 (Conv2D)	(None, 64, 64, 64)	18496
max_pooling2d_1 (MaxPooling2D)	(None, 32, 32, 64)	0
conv2d_3 (Conv2D)	(None, 32, 32, 128)	73856
max_pooling2d_2 (MaxPooling2D)	(None, 16, 16, 128)	0
dropout (Dropout)	(None, 16, 16, 128)	0
flatten (Flatten)	(None, 1048576)	0
dense (Dense)	(None, 64)	6422592
dropout_1 (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 1)	65

Total params: 6,528,097
Trainable params: 6,528,097
Non-trainable params: 0

```
[34]: #Compile our model
model.compile(optimizer = 'adam', loss = keras.losses.binary_crossentropy, metrics = ['accuracy'])
```

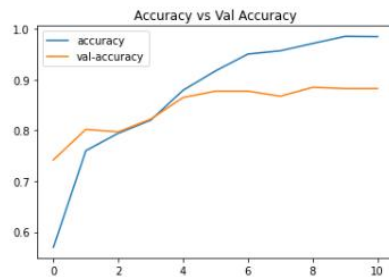
```
[35]: #Early stopping and model checkpoint
from keras.callbacks import ModelCheckpoint, EarlyStopping

es = EarlyStopping(monitor = 'val_accuracy', min_delta = 0.01, patience = 5, verbose = 1, mode = 'auto')
mc = ModelCheckpoint(monitor = 'val_accuracy', filepath = './bestmodel.h5', verbose = 1, save_best_only = True, mode = 'auto')

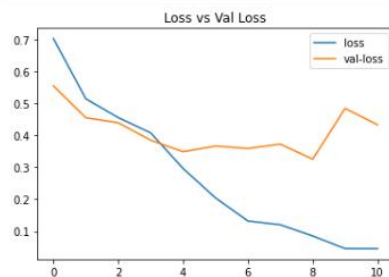
cd = [es, mc]
```

```
[36]: #Train our model
hist = model.fit(x = X_train, y = y_train, batch_size = 32, epochs = 30, validation_data = (X_val, y_val), callbacks = cd, verbose = 1)
```

```
[37]: #Plot the graphical interpretation
h = hist.history
plt.plot(h['accuracy'], label = 'accuracy')
plt.plot(h['val_accuracy'], label = 'val-accuracy')
plt.title('Accuracy vs Val Accuracy')
plt.legend()
plt.show()
```



```
[38]: #Plot the graphical interpretation
h = hist.history
plt.plot(h['loss'], label = 'loss')
plt.plot(h['val_loss'], label = 'val-loss')
plt.title('Loss vs Val Loss')
plt.legend()
plt.show()
```



```
[39]: #Test our model on the test set
from keras.models import load_model
model = load_model('/content/bestmodel.h5')
acc = model.evaluate(X_test, y_test)[1]
print(f'The accuracy of our model is {acc}')

13/13 [=====] - 1s 54ms/step - loss: 0.3359 - accuracy: 0.8947
The accuracy of our model is 0.8947368264198303
```

We can see that our model works well on the test set, so it's not overfitted

```
[75]: #Try our model on a random image that it has never seen before
from keras.preprocessing.image import load_img, img_to_array
#Choose a MRI image with tumor
path = "/content/aug_data/yes/aug_Y53_0_8799.jpg"
img_yes = load_img(path, target_size=(240,240))
img_array_yes = img_to_array(img_yes)/255
img_array_yes = np.expand_dims(img_array_yes, axis=0)
prediction1 = model.predict(img_array_yes)
class1 = np.round(prediction1).astype(int)
if class1 == 0:
    print("The MRI image doesn't have a Tumor")
else:
    print("The MRI image has a tumor")

The MRI image has a tumor
```

```
[76]: #Try our model on a random image that it has never seen before
from keras.preprocessing.image import load_img, img_to_array
#Choose a MRI image without tumor
path = "/content/aug_data/no/aug_11 no_0_8546.jpg"
img_no = load_img(path, target_size=(240,240))
img_array_no = img_to_array(img_no)/255
img_array_no = np.expand_dims(img_array_no, axis=0)
prediction2 = model.predict(img_array_no)[0][0]
class2 = np.round(prediction2).astype(int)
if class2 == 0:
    print("The MRI image doesn't have a Tumor")
else:
    print("The MRI image has a tumor")

The MRI image doesn't have a Tumor
```


▼ Transfer Learning ¶

```
[85]: #Import MobileNet Model
from keras.applications.mobilenet import MobileNet
base_model = MobileNet(input_shape=(240,240,3), include_top=False)

WARNING:tensorflow:'input_shape' is undefined or non-square, or 'rows' is not in [128, 160, 192, 224]. Weights for input shape (224, 224) will be loaded
as the default.

[86]: #Make the Layers of the model untrainable
for layer in base_model.layers:
    layer.trainable = False

[87]: #Add a Flatten and dense Layers to the base model
X = Flatten()(base_model.output)
X = Dense(units=1, activation='sigmoid')(X)
transfer_model = Model(base_model.input, X)

[ ]: transfer_model.summary()

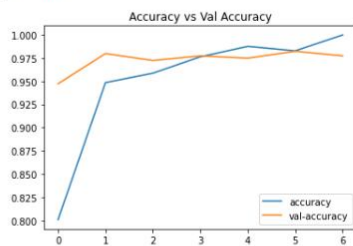
[89]: #Compile the new model
transfer_model.compile(optimizer='rmsprop', loss=keras.losses.binary_crossentropy, metrics=['accuracy'])

[90]: #Early stopping and model checkpoint
est = EarlyStopping(monitor='val_accuracy', min_delta = 0.01, patience = 5, verbose = 1, mode = 'auto')
mct = ModelCheckpoint(monitor='val_accuracy', filepath = './besttransfermodel.h5', verbose = 1, save_best_only = True, mode = 'auto')

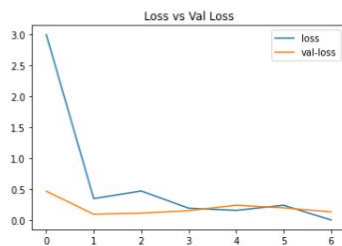
cdt = [est, mct]

[91]: #Fit our new model
transfer_hist = transfer_model.fit(x = X_train, y = y_train, batch_size = 32, epochs = 30, validation_data = (X_val, y_val), callbacks = cdt, verbose = :
```

```
[92]: #Plot the graphical interpretation
h = transfer_hist.history
plt.plot(h['accuracy'], label = 'accuracy')
plt.plot(h['val_accuracy'], label = 'val-accuracy')
plt.title('Accuracy vs Val Accuracy')
plt.legend()
plt.show()
```



```
[93]: #Plot the graphical interpretation
h = transfer_hist.history
plt.plot(h['loss'], label = 'loss')
plt.plot(h['val_loss'], label = 'val-loss')
plt.title('Loss vs Val Loss')
plt.legend()
plt.show()
```



```
[94]: #Test our model on the test set
from keras.models import load_model
model = load_model('./content/besttransfermodel.h5')
acc = model.evaluate(X_test, y_test)[1]
print(f'The accuracy of our model is {acc}')

13/13 [=====] - 2s 90ms/step - loss: 0.0711 - accuracy: 0.9875
The accuracy of our model is 0.9874686598777771
```

3.Data Acquisition and Analysis

Biology and medicine data are not as direct and meaningful as physical signals. With the advancement in the technology, data from biological specimens could be captured directly or indirectly by sensors. The details derived from the data could then be used for analysis, diagnosis, and treatment. The method of sampling signals to calculate real-world physical conditions and converting the resulting samples into digital numeric values that can be manipulated by a computer is known as data acquisition. Whereas, the compilation and manipulation of data to generate useful results is referred to as data processing.

Electroencephalography (EEG) is one of the techniques for collecting data from the human brain. It was developed in the 1930s by Hans Berger, a German psychiatrist. It is a noninvasive approach for detecting and recording brain electrical activity using electrodes connected to the scalp that track variations in electric potential on the skin surface caused by the activity of cerebral neurons and then amplify them to form a record (an encephalogram). Neurologists now use EEG to distinguish between functional and organic brain conditions, diagnose sleep disturbances, headaches, and to control brain activity throughout cardiac operations. The disadvantages of using an electroencephalograph in use include the equipment's limited resolution and the ability to display and interpret data on a screen.

The second method is magnetoencephalography (MEG). MEG is a technique for measuring the magnetic field generated by the human brain. It allows for a much higher spatial precision signal and interpretation over a much broader frequency spectrum than EEG. The behavior of the neuron population parallel to the scalp is much more receptive to the MEG signal than it is perpendicular. MEG is used in biomedical experiments to assess the roles of specific brain areas, as well as in clinical diagnostics and as a tool for locating abnormal regions during neurosurgical procedures.

The third type of magnetic resonance imaging is functional magnetic resonance imaging (fMRI), which detects an increase in blood supply and oxygenation in the active portion of the brain. fMRI is focused on

the application of magnetic resonance imaging (MRI) and its extension through observation based on the properties of oxygenated and deoxygenated blood. The use of continuous magnetic field gradients to register these waves (caused by protons returning to their ground state emit an electromagnetic) allows a device to reconstruct the representation of the interior of the object under analysis.

Fourth, positron emission tomography (PET) is a procedure for imaging that records the radiation released during positron annihilation. The registered data is saved on a storage disk in digital form, allowing for the construction of cross-sectional photographs of the patient's body, similar to those produced by MRI. Currently, almost all positron emission tomography scanners on the market are hybrid instruments of the type: PET-CT, PET/CT—PET with a multi-row computed tomography scanner PET-MRI, also known as PET/MRI, is a hybrid of PET and magnetic resonance imaging.

Finally, near infrared spectroscopy (NIRS) is a method for visualizing brain function that involves sending laser beams across the skull. Blood that has absorbed oxygen receives light waves at different frequencies than blood that has not absorbed oxygen. Researchers can monitor blood pressure by measuring the amount of light transmitted from the brain at different wavelengths. Diffuse optical tomography, or DOT, is the procedure used, if the purpose was to create the activation map. Whereas for registration purpose, it is an event based optical signal (EROS) that registers light diffusion due to shifts in cells that arise during the excitement of neurons. Although techniques like diffuse optical tomography and NIRS rely on blood flow to measure optical absorption of hemoglobin, EROS uses the scattering properties of neurons to provide a much more precise measure of cellular activity.

These modern data acquisition techniques focusing on brain impulses include EEG, NIRS, fMRI, and PET have a readout of the messages in the human brain, as well as a method for archiving and interpreting them as. A multi-channel encephalograph was used to demonstrate the signal readout. Time-varying EEG signals from individual electrodes were captured in the .edf format using the Emotiv Xavier TestBench program, and then subjected to Toolbox EEGLab for Matlab.

After training the model and segmentation, as mentioned in the materials and methodology section, all cases with tumors were subjected to heuristic rules over the deep learning-based segmentation model and features such as tumor signal intensity on T1WI, T2WI, FLAIR, enhancement pattern, cyst, necrosis, haemorrhage, and diffusion restriction were predicted for all cases. The diagnostic accuracy of feature prediction by the deep learning-based model as opposed to the radiologist detected MRI findings is shown in Table 3.

Diagnostic accuracy of feature prediction by deep learning based model as opposed to radiologist detected MRI findings					
Predicted feature <i>n</i> = 70	Sensitivity	Specificity	PPV	NPV	Accuracy
Hypointensity on T1WI	86.36%	0	93.44%	0	81.43%
Isointensity on T2WI	63.16%	84.62%	94.74%	34.37%	67.14%
Hyperintensity on FLAIR	85.71%	61.9%	60%	86.67%	71.4%
Haemorrhage	54.17%	78.26%	56.62%	76.6%	70%
Cyst	79.07%	59.26%	75.56%	64%	71.4%
Necrosis	52.94%	83.02%	50%	84.62%	75.71%
Enhancement heterogeneity	81.36%	72.73%	94.12%	42.11%	80%
Enhancement quantification	Mild	50%	92.86%	82.35%	73.58%
	Moderate	61.1%	71.15%	42.31%	84.09%
	Severe	83.3%	84.78%	74.07%	90.70%
Diffusion restriction	53.33%	96.77%	94.12%	68.16%	75.4%

[Open in a separate window](#)

n: total number of abnormal cases which were picked by AS model and on which the mentioned features are predicted by AS model

Table 3

Conclusion

This review is to provide a detailed summary of the current state of pediatric brain tumors studies centered on medical imaging based on deep learning. Due to many challenges associated with this domain, there were relative scarcity of publications of deep learning-based studies of pediatric brain tumor images. Only a recent similar review study in this field was conducted a year ago, however, the emphasis was only on presenting deep teaching methods in infant MRI systems only in the segmentation of infant brain tissue at the isointense phase and pre-symptomatic condition predictive autism spectrum disorder (ASD).

References

- <https://www.cancer.org/cancer/brain-spinal-cord-tumors-children/about/types-of-brain-and-spinal-tumors.html>
- <http://www.developingconnectome.org/project/>
- <https://www.humanconnectome.org/article/data-release-10-available-hcp-lifespan-aging-and-development>
- <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC8230188/#B37-brainsci-11-00716>
- <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC10501890/#:~:text=Data%20was%20divided%20into%20training,and%20validation%20data%2C%209%20cases.>
- https://scholar.google.com/scholar_lookup?title=Recommendations+for+Cross-Sectional+Imaging+in+Cancer+Management&author=D.M.+Byrne&publication_year=2014&
- <https://onlinelibrary.wiley.com/doi/abs/10.1111/j.1750-3639.1993.tb00752.x>